Reproducing LEBench on a Virtual Machine

An Analysis of Performance Evolution of Linux's Core Operations

Haoyu Zou 5260000157
CS340 Operating Systems, 2026–2027

9 November 2026

1 Introduction and claims

Ren et al. measured the latency of Linux core operations from 3.0 through 4.20 and argued that the kernel got slower, mostly because of security work, new features, and a few configuration mistakes [1]. The operations are the ones LEBench times directly: `getpid`, context switch, fork, read/write, mmap/munmap, page fault, send/recv, and select/poll/epoll.

Three claims are tied to the paper:

1. **Figure 1(a).** Relative to Linux 4.0, later kernels are slower on these operations, except big write and big munmap. Most tests are also slower than on 3.0, and many are at least 50% slower.
2. **Table 3 and Figure 2.** Eleven changes explain the large jumps. Four are security patches, four are features, and three are configuration mistakes.
3. **Section 5.** Turning those eleven changes off speeds Redis, Apache, and Nginx by up to 56%, 33%, and 34%.

This reproduction tests claim 1 on the kernel list shipped with the authors' experiment notes, and tests one factor from claim 2 (page-table isolation). Claim 3 is tested as one runtime switch, `spectre_v2=off`, on the guest's recovery kernel. The authors' `EXPERIMENTS.md` lists 41 Ubuntu mainline builds, from 3.0.7 to 4.20.13. The paper text says 36 versions. The driver attempts every release in that file.

2 Reproduction setup

2.1 Starting code

The benchmark is the authors' LEBench tree at commit `6aa7801` (`https://github.com/LinuxPerfStudy/LEBench`), Apache-2.0, copyright 2019 University of Toronto. `TEST_DIR/OS_Eval.c`, `run.py`, and `get_kern.py` are unchanged. The course runner is `course/OS_Eval_reduced.c`, generated from that file by `course/make_reduced_eval.py`. It keeps the same `clock_gettime` intervals and the same K-best rule (five samples within 5%). `BASE_ITER` is 40 instead of 10,000, the 60-second settle is omitted, and three cases that either block or dominate runtime are compiled out: the 96 KB send/recv pair, the 12,000-page fork, and the 10,000-page file tests. A null K-best result, which aborts the original program on a noisy virtual machine, falls back to the smallest sample.

A separate rewrite that reimplemented the tests with `rdtsc` was not used. Section 6 says why.

2.2 Environment

Item	Setting
Host	Windows 11, 13th Gen Intel Core i7-13700H
Hypervisor	VirtualBox 7.2.22, legacy BIOS, NAT
Guest	Ubuntu 16.04.6 LTS, English
Resources	2 virtual CPUs, 4 GB RAM, 80 GB disk on `D:`
Compiler	gcc 5.4.0 (`-O0 -pthread`, the upstream makefile flags)
Known-good kernel	4.4.0-210-generic, used only to install and recover

Memory integrity is enabled on the host, so VirtualBox cannot use its fast hypervisor path. The guest CPU model string is still the host's i7-13700H. That CPU reports Meltdown as "Not affected".

2.3 Workloads

Each booted kernel runs `course/sweep_bench` three times as root. The binary is the reduced `OS_Eval`. It writes the authors' CSV: one K-best row and one average row per test, in seconds. The reported number is the mean and sample standard deviation of the three K-best values. Logs are `results/kernels/<release>/runN.csv` plus stdout, stderr, the exit code, and `environment.txt`.

Kernel packages come from the authors' backup, `https://github.com/LinuxPerfStudy/ExperimentSetup`, directory `default_ubuntu_kernels`. The driver downloads the `linux-image` deb (the `unsigned` package where that is the only image), installs it, and selects it for one boot with `grub-reboot`. A panic returns to 4.4.0-210 on the next power-on. It does not run the upstream `grub-install` against a partition.

2.4 Deviations

Item	Paper	Used here	Reason
Machine	HP DL160 G9, Xeon E5-2630 v3, 128 GB, SSD	VirtualBox on an i7-13700H, 2 vCPU, 4 GB	The server is not available. Comparisons stay on one guest.
Kernel count	36 versions in the paper text	All 41 releases listed by the authors	The released list is the reproducible set.
Iterations	`BASE_ITER` 10,000 and a 60 s settle	`BASE_ITER` 40, no settle	A full pass is hours per kernel on this guest. The K-best rule is unchanged.
Omitted tests	96 KB send/recv, 12,000-page fork, 10,000-page I/O	Not compiled in	The upstream comment says the large send blocks on old kernels. The other two dominate runtime.
Statistics	K-best inside one process	That value, then mean ± sample standard deviation over 3 processes	The course requires the cross-run statistic.
Applications	Redis, Apache, Nginx, all 11 causes disabled, many kernels	Redis 4.0.8, Apache 2.4.18, Nginx 1.10.3, one kernel, only `spectre_v2=off`	The guest can toggle one mitigation. The other ten causes need kernel rebuilds.

3 Results

3.1 Page-table isolation on the installed kernel

This comparison changes one factor. Kernel 4.4.0-210-generic is booted twice: the default command line, and the same kernel with `nopti`. Hardware, binary, and iteration count stay the same. `/sys/devices/system/cpu/vulnerabilities/meltdown` reads "Not affected" in both boots. Spectre v2 stays on Enhanced IBRS.

Table 1 is computed by `scripts/render_results.py` from `results/kernel_summary.csv`. "Overlap" means the two mean±stdev intervals touch.

Table 1: Same kernel and binary, default command line versus `nopti`. Three runs each.

Test	Default	*nopti*	Change	Intervals overlap?
getpid	314 ns	295 ns	-6%	yes
context siwtch	11.2 us	12.9 us	+15%	yes
send	600 ns	693 ns	+15%	yes
recv	692 ns	764 ns	+10%	yes
fork	92.8 us	254.0 us	+174%	no
small read	784 ns	946 ns	+21%	yes
small write	1.9 us	2.0 us	+7%	yes
big write	1.66 ms	3.62 ms	+118%	yes
big munmap	269.5 us	266.3 us	-1%	yes
select	1.2 us	1.5 us	+29%	yes
poll	847 ns	1.1 us	+26%	yes
epoll	905 ns	1.0 us	+12%	yes

Fast operations move by less than their run-to-run spread. `fork` is slower with `nopti`, and the intervals do not overlap. That is the opposite of the paper's KPTI tax, and it is not a clean ablation: this CPU does not enable Meltdown page-table isolation in the first place.

3.2 Kernel sweep

Of the 41 releases in `EXPERIMENTS.md`, 34 booted far enough for three runs. Kernels 3.0.7 through 3.5.4 install and then never answer sshd; they are listed in `results/kernels/boot-failures.txt` and have no latency. Linux 4.19.11 needs a `linux-modules` package that is no longer published at the Ubuntu mainline URL used for 4.16, 4.17, 4.18, and 4.20, so that one release was not installed. The 4.4.0-210 pair is the guest's recovery kernel and is not one of the 41.

Across 627 later-kernel measurements, 271 (43%) have a higher mean than the same test on 4.0.5 and 76 (12%) have a mean at least 50% higher. 117 of those measurements have non-overlapping mean±stdev intervals and a higher mean, which is the bar used in the analysis.

Those 627 rows are every test on the 19 kernels newer than 4.0.5 that finished three runs. A zero K-best on 3.6.10 (getpid and a few other tests) is a failed window: the average in the same CSV is not zero, and 3.6.10 is not the baseline, so it is not in the 627. Table 3 shows three tests. Every mean is the mean of three K-best values in `results/kernels/<release>/run{1,2,3}.csv`, and the full set is `results/kernel_summary.csv`.

Table 2: Paper versus this guest. The metric is the sign of the latency change from Linux 4.0, which is what Figure 1(a) plots.

Item	Paper	Ours
Later kernels vs 4.0	Most tests slower, and many by at least 50%	271 of 627 means (43%) are higher; 76 (12%) are at least 50% higher
vs Linux 3.0	75% of tests slower	No measurement: 3.0.7–3.5.4 never reached sshd

Table 3: Mean ± sample standard deviation of three K-best runs.

Kernel	getpid	big write	big munmap
3.6.10	0±0 ns	1.42±0.12 ms	220.4±190.9 us
3.7.9	422±61 ns	2.63±1.18 ms	273.1±6.7 us
3.8.10	317±44 ns	1.91±0.44 ms	403.2±168.1 us
3.9.8	295±18 ns	2.70±1.34 ms	459.6±272.4 us
3.10.10	921±23 ns	2.26±0.24 ms	329.0±50.8 us
3.11.6	325±43 ns	2.04±0.42 ms	379.1±58.3 us
3.12.8	1.0±0.1 us	2.65±1.66 ms	350.1±61.8 us
3.13.7	1.1±0.2 us	1.63±0.07 ms	321.0±10.7 us
3.14.6	878±26 ns	2.32±0.15 ms	373.3±48.1 us
3.15.8	998±126 ns	2.38±0.56 ms	340.3±87.3 us
3.16.3	322±41 ns	1.09±0.46 ms	178.7±83.5 us
3.17.6	238±75 ns	1.54±0.68 ms	218.6±8.5 us
3.18.6	293±37 ns	2.93±1.42 ms	228.9±29.7 us
3.19.3	253±6 ns	2.08±0.40 ms	200.3±0.5 us
4.0.5	245±0 ns	1.66±0.28 ms	199.7±3.5 us
4.1.6	255±6 ns	1.95±0.50 ms	193.6±8.1 us
4.2.5	248±9 ns	1.37±0.57 ms	161.7±55.2 us
4.3.3	224±93 ns	1.67±0.55 ms	157.4±54.9 us
4.4.5	238±8 ns	1.65±0.10 ms	198.6±5.9 us
4.5.4	189±0 ns	1.59±0.15 ms	166.4±50.7 us
4.6.4	117±62 ns	1.67±0.94 ms	184.9±87.7 us
4.7.6	192±1 ns	4.24±2.97 ms	334.2±100.0 us
4.8.14	225±29 ns	2.87±1.62 ms	210.6±0.5 us
4.9.11	203±85 ns	3.72±1.20 ms	480.8±415.3 us
4.10.13	197±6 ns	1.71±0.15 ms	206.9±3.8 us
4.11.8	212±28 ns	1.63±0.40 ms	335.8±236.1 us
4.12.10	195±9 ns	1.58±0.27 ms	247.3±58.3 us
4.13.12	251±10 ns	1.90±0.73 ms	211.4±2.6 us
4.14.15	535±12 ns	1.77±0.21 ms	206.0±6.4 us
4.15.15	248±30 ns	2.55±0.64 ms	212.3±39.9 us
4.16.13	100±0 ns	657.6±46.5 us	83.6±0.8 us
4.17.14	100±1 ns	611.1±12.2 us	84.0±1.0 us
4.18.16	100±0 ns	671.0±17.4 us	79.5±0.7 us
4.20.13	230±44 ns	2.27±0.47 ms	192.7±3.9 us

4 Analysis

Claim 1 is about the sign of the change from 4.0, not the absolute nanoseconds. Absolute times here are not the paper's times: the guest has two slow virtual CPUs, the disk is a virtual disk, and the iteration count is shorter, so a single outlier moves the K-best window. The standard deviations in Table 1 are often a large fraction of the mean. A difference smaller than the standard deviation is not evidence. The verdict below uses that rule: a later kernel counts as slower only when its

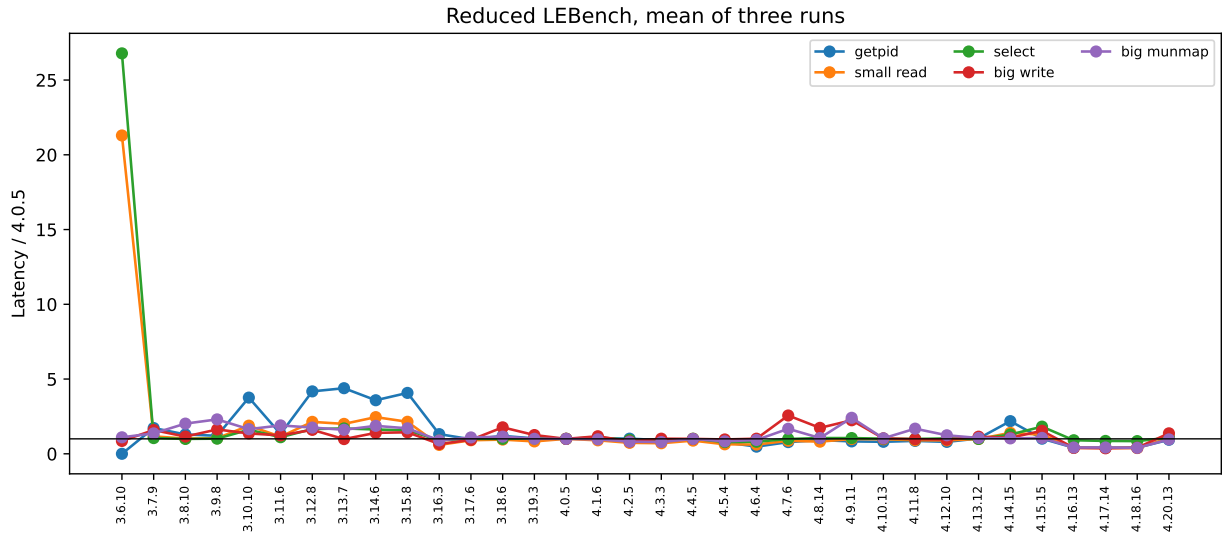


Figure 1: Latency divided by the 4.0.5 result on this guest. The line at 1 is the paper's baseline kernel.

getpid has a higher mean on 5 and a clearly faster interval on 9; fork has a higher mean on 0. Big write, which the paper treats as an exception that got faster, has a higher mean on 11 of 19 kernels, but only 1 of those intervals clears the noise. The comparison with Linux 3.0 is **inconclusive**: 3.0.7 through 3.5.4 never reached sshd, so there is no 3.0 baseline on this guest.

Claim 2 is only touched by the `nopti` switch. The paper's KPTI result needs a CPU that pays for page-table isolation. The i7-13700H reports that it does not. The switch still changes `fork`, so the command line is not a no-op, but it does not reproduce "KPTI makes `getpid`, read, and recv slower." That part of claim 2 is **not supported** here.

Claim 3 was run on 4.4.0-210-generic only. The paper turns off eleven causes and reports speed-ups of up to 56%, 33%, and 34% for Redis, Apache, and Nginx. Here the only change is `spectre_v2=off`. The vulnerability file moves from "Mitigation: Enhanced IBRS" to "Vulnerable", so the switch took effect. Table 4 is mean ± sample standard deviation of three runs, in requests per second. Logs are `results/macro/<config>/results.csv`.

Table 4: Application throughput on one kernel. Ratio is the `spectre_v2=off` mean divided by the default mean.

Test	Default	`spectre_v2=off`	Ratio
Apache index	1580 ± 139 /s	1308 ± 24 /s	0.83
Nginx index	5725 ± 966 /s	7314 ± 208 /s	1.28
Redis GET	36386 ± 6116 /s	49172 ± 14833 /s	1.35
Redis RPUSH	29938 ± 7011 /s	37689 ± 4546 /s	1.26
Redis RPOP	32218 ± 9314 /s	37221 ± 18717 /s	1.16
Redis SPOP	27424 ± 6284 /s	32762 ± 3120 /s	1.19
Redis LRANGE 100	13657 ± 2430 /s	14687 ± 2755 /s	1.08

Apache is slower with the switch, and the intervals do not overlap (1580 ± 139 against 1308 ± 24). Nginx is faster, and those intervals do not overlap either (5725 ± 966 against 7314 ± 208).

Every Redis row has a higher mean, and every Redis interval still overlaps the default interval. The three applications do not move in the same direction, and the Redis changes are inside the run-to-run spread. Claim 3 is **not supported** on this guest. The paper's percentages come from disabling eleven causes on a server, not one command-line switch on this virtual machine.

A separate audit of the existing sweep, in `docs/timer-floor.md`, asks whether claim 1 is an artifact of the clock. It treats `ref` as the timer floor and drops a row when the mean is below three times that floor, or when the floor is zero. That removes 56 of 1152 rows (4.9%), mostly `getpid` and the whole 3.6.10 boot. The median ratio to 4.0.5 stays 1.02, and the share of rows at least 50% higher stays near 18%. The 4.16.13–4.18.16 rows look about twice as fast, including the pure-CPU loop, which does not call the kernel. Those three boots are a shifted time base, not a faster kernel. Dropping the timer-bound rows does not turn claim 1 into the paper's result.

What this run cannot show is the heatmap on the original server or the eleven-way ablation. Kernels that do not reach sshd are recorded as boot failures in `results/kernels/progress.log` and are not given a latency.

5 Extension

The reduced suite times one `getpid` between two `clock_gettime` calls. On 4.0.5 that empty pair, the `ref` row, is 156 ± 64 ns, and the `getpid` row is 245 ± 0 ns, so the timer is a large part of the number. The extension changes only that test. `course/OS_Eval_batched.c` is the reduced source with `getpid` rewritten to time 1,000 calls and divide by 1,000. K-best, `BASE_ITER`, the compiler flags, and every other test stay the same. Each kernel ran the new binary three times. Logs are `results/batching/<release>/run{1,2,3}.csv`.

Table 5: Mean ± sample standard deviation of three K-best runs. Single-shot numbers are the reduced sweep; batched numbers are the one-line change above.

Kernel	Single-shot getpid	Batched getpid
4.0.5	245 ± 0 ns	129 ± 0 ns
4.4.0-210	314 ± 39 ns	127 ± 40 ns

On 4.0.5 the batched per-call cost is 129 ns, against 245 ns for one call. The two intervals do not overlap. The same binary's empty clock pair that day was 132 ± 19 ns, close to the batched per-call figure, which is what a timer-dominated sample looks like. On 4.4.0-210 the single-shot mean is 314 ns and its interval sits entirely above 4.0.5. After batching, the means are 127 ns and 129 ns, and the intervals overlap. The apparent `getpid` slowdown between these two kernels does not survive the change. `small read` on the batched binary stays within the single-shot spread (4.0.5: 813 ± 72 ns against 726 ± 73 ns), so the rest of the suite did not move with this edit.

6 The other implementation

A second tree, written as a from-scratch LEBench, lives outside this repository. It is a Linux C program (`src/bench_*.c`) plus a Chinese report whose result table is empty. It was not used for the numbers above. The useful ideas in it are the K-best description and the list of operations. The measurement itself should not be substituted for `OS_Eval`:

- It times with `rdtsc` and does not serialize the instruction stream. On this VirtualBox

guest the TSC is not a stable offset from the host, which is why the authors' code uses `clock_gettime`.

- `select` allocates and fills the descriptor bitmap inside the timed region, so the number includes allocator traffic.
- Context switch is one long loop divided by the iteration count. That is an average, not the paper's K-best of repeated trials, and it never raises the priority to −20.
- The page-fault sample touches every page and divides by the page count. Fault-around fills neighboring pages on the first miss, so later touches in that sample are often not faults. `OS_Eval` times one touch.
- Iteration counts, the empty-syscall choice, and the lack of a third process run all differ from the paper and from the course rule.
- It was checked by compiling against stand-in headers on Windows. It had not been executed on Linux, and the report says so.

The changes that make the course tree more faithful are the ones in `course/` and `scripts/vm_kernel_sweep.py` call the authors' functions, keep K-best, repeat three times, create the Unix-socket directory `OS_Eval` concatenates onto `LEBENCH_DIR`, and boot with `grub-reboot` instead of installing GRUB onto a partition.

7 Conclusion

The reproduction is partial. Thirty-four of the 41 listed kernels ran the reduced benchmark three times. On those logs, the claim that most core operations got slower after 4.0 is not supported, and removing timer-bound rows does not reverse that. The `nopti` switch does not reproduce the KPTI slowdown on a CPU that is not subject to Meltdown. Batching `getpid` on 4.0.5 and 4.4.0-210 cuts the reported per-call cost by about half. Turning Spectre v2 off speeds Nginx and slows Apache, while Redis stays inside its run-to-run spread, so the application claim is not supported either. The eleven-cause ablation is not reproduced. The practical lesson is that a microbenchmark copied onto a new timer, and a kernel image that does not boot, both fail quietly: the first still prints a number, and the second never gets far enough to print one. The logs have to show which of those happened.

Contribution statement

Haoyu Zou (5260000157) imported LEBench at `6aa7801`, built the Ubuntu 16.04.6 guest, wrote the reduced runner and the reboot driver, collected the logs under `results/kernels/`, measured the batched-`getpid` variant under `results/batching/`, ran the Redis, Apache, and Nginx comparison under `results/macro/`, and wrote this report and the slides. Commits for this work use that student ID and name.

AI usage statement

An AI coding assistant in Cursor drafted the runner, the reboot script, this report, and the slides, and operated the virtual machine. Kernel packages, the benchmark source, and the latency numbers are from the LEBench authors and from the guest logs. The assistant did not invent measurements.

References

[1] Xiang (Jenny) Ren, Kirk Rodrigues, Luyuan Chen, Camilo Vega, Michael Stumm, and Ding Yuan. An Analysis of Performance Evolution of Linux's Core Operations. In *Proceedings of SOSP*, 2019. https://doi.org/10.1145/3341301.3359640